

Backend Discovery

PHASE 0

Goal

Understand the backend.

No frontend.

Deliverables

- Router analysis
 - API mapping
 - JWT flow
 - RAG flow
 - Services
 - Database
 - Docker
 - Backend documentation
-

PHASE 1

Frontend Architecture

Still no React.

Now design

- Folder structure
- Feature modules
- Routing
- State
- Services
- API layer
- Authentication
- Theme
- Responsive strategy

Deliverable

Frontend Architecture Document

PHASE 2

Foundation

Now coding starts.

Create



Nothing visual.

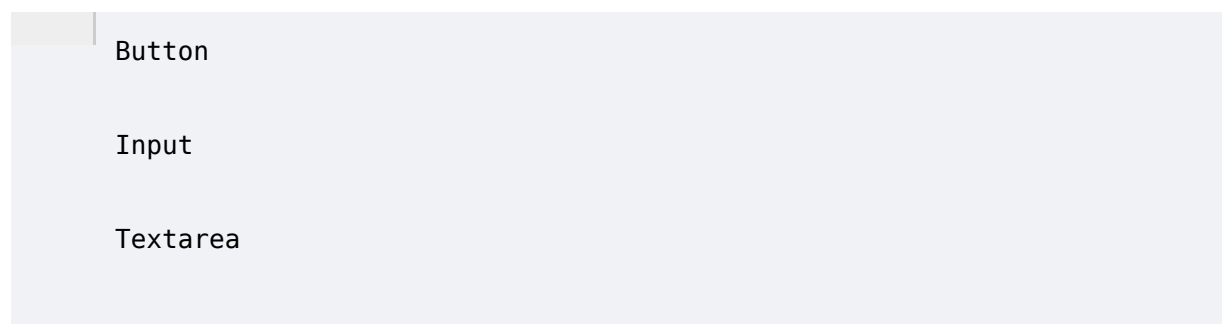
PHASE 3

Design System

Build reusable UI.

Only components.

Example



Card

Badge

Avatar

Dropdown

Dialog

Modal

No pages.

PHASE 4

Authentication

Build

Login

JWT

Axios

Protected Routes

Current User

Logout

Session Restore

Now user enters application.

PHASE 5

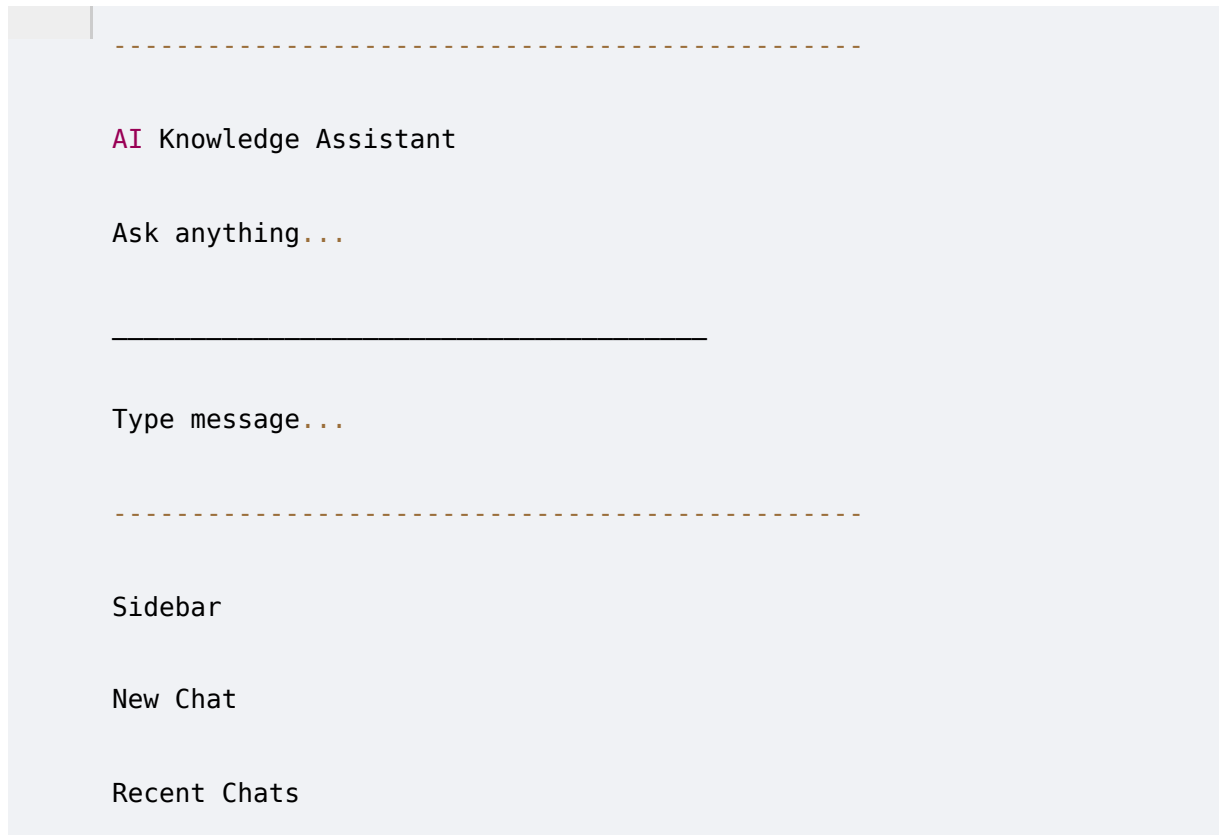
Application Shell

This is where your idea starts.

Instead of Dashboard...

The first page becomes

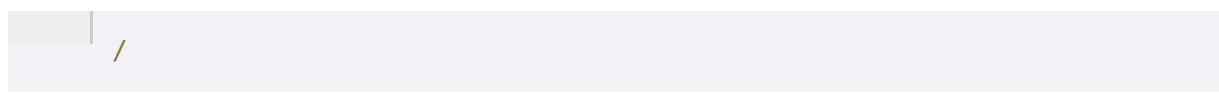
AI Workspace



Exactly like ChatGPT.

No dashboard.

This becomes



PHASE 6

AI Chat Experience

Biggest phase.

ChatGPT + Claude.

Features

- ✓ Markdown
- ✓ Code Highlight
- ✓ Copy

- ✓ Regenerate
- ✓ Timestamp
- ✓ Typing
- ✓ AI Avatar
- ✓ User Avatar
- ✓ Scroll
- ✓ Keyboard Shortcuts
- ✓ Streaming ready

Future ready.

PHASE 7

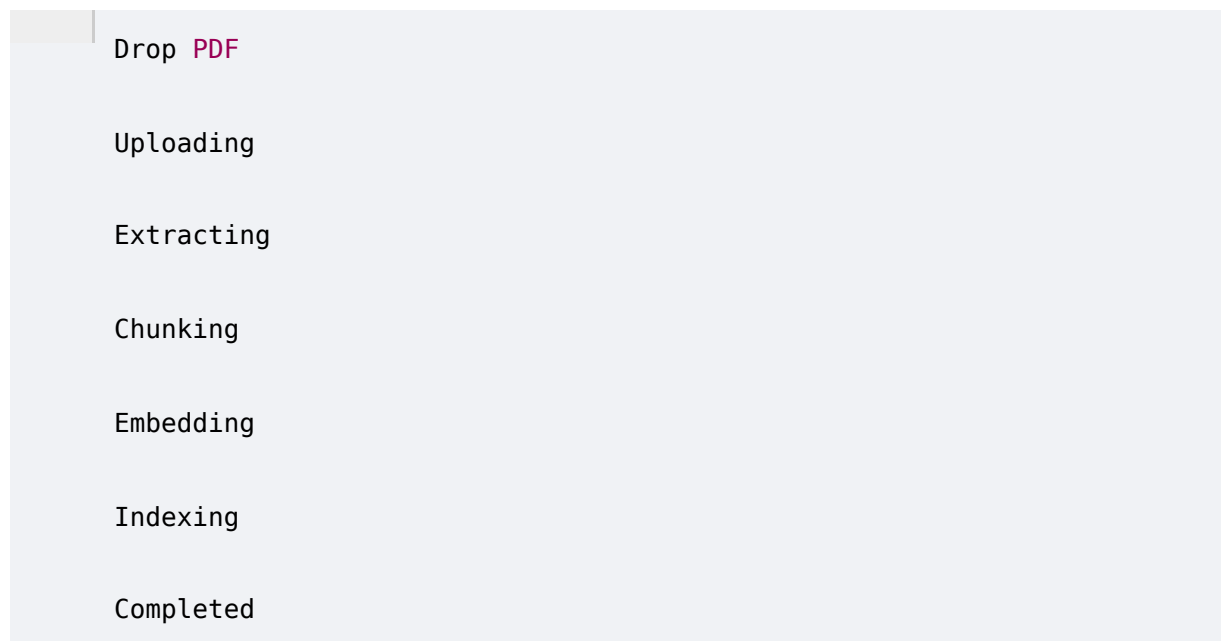
Knowledge Base

Beautiful upload.

Not

Choose File

Instead



Animated.

PHASE 8

Documents

Cards.

Python.pdf
Uploaded
Indexed
Chunks
Delete
View

Search.

Filter.

Pagination.

PHASE 9

RAG Experience

One of the coolest phases.

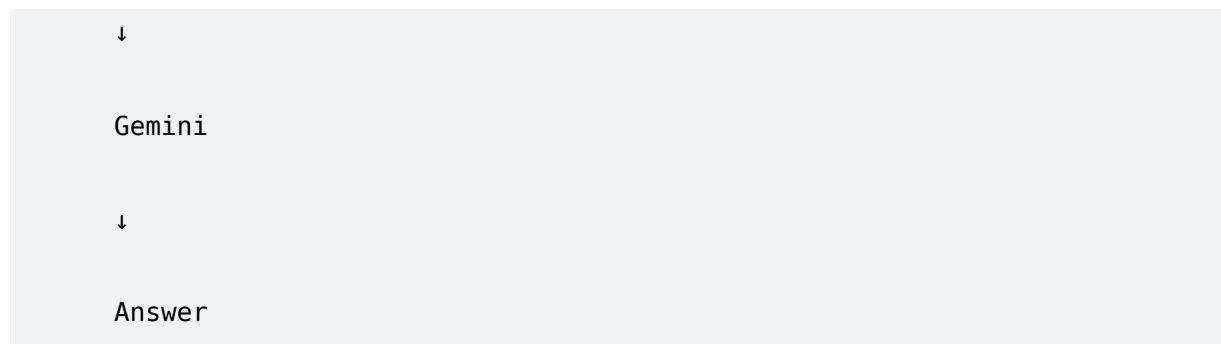
Instead of

Answer

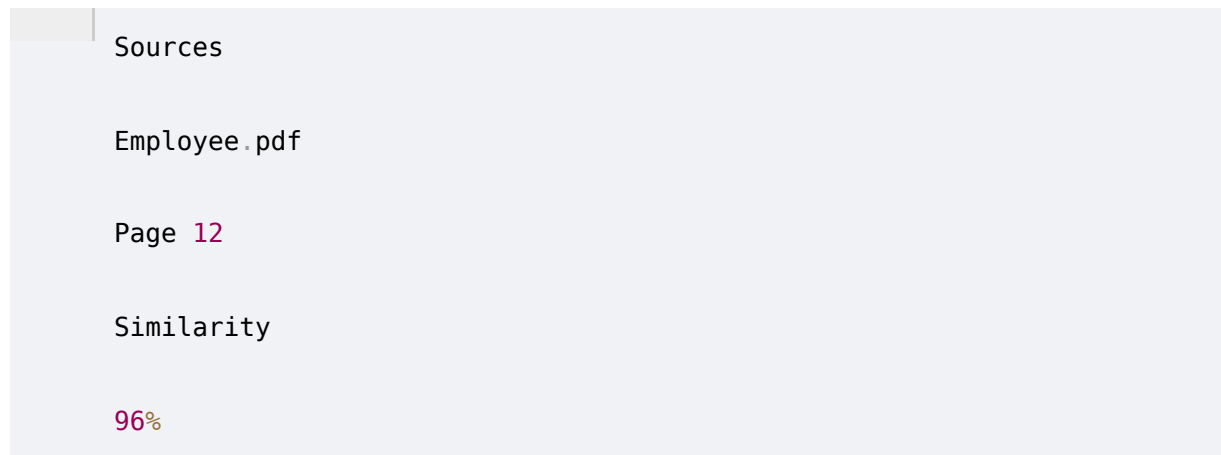
Only

Show

Question
↓
Retriever
↓
Relevant Chunks



and



Recruiters immediately understand

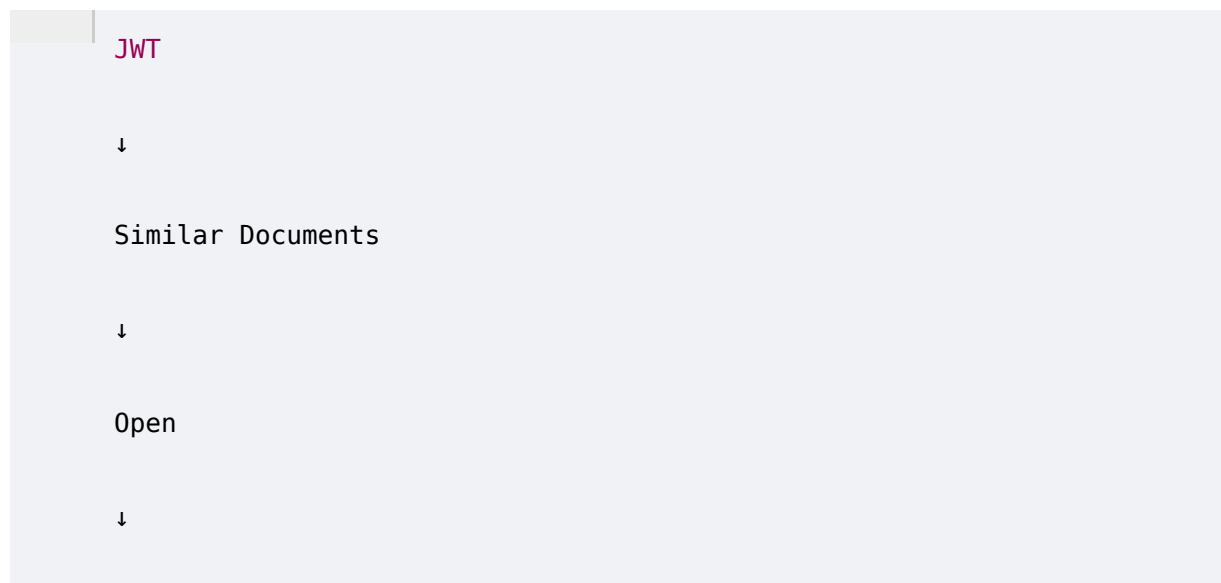
This is RAG.

PHASE 10

Semantic Search

Use Search API.

Beautiful.



Preview

PHASE 11

Dashboard

Notice

Dashboard is NOT first.

It comes much later.

Minimal.

Chats

Documents

Embeddings

Cache

Activity

PHASE 12

Analytics

I actually want to add this.

Because your backend already has enough data.

Show

- Total chats
 - Total uploads
 - Recent activity
 - Cache hits (if available)
 - AI usage (if available)
-

PHASE 13

Profile

Settings

Theme

Gemini Model

API Status

Logout

PHASE 14

Premium UX

Everything.

Animations.

Micro interactions.

Hover.

Glass.

Loading.

Skeleton.

Empty State.

Command Palette.

Keyboard Shortcuts.

Responsive.

Accessibility.

PHASE 15

Optimization

Lazy Loading

Code Splitting

Performance

Caching

Bundle

PHASE 16

Production Ready

README

Deployment

Screenshots

Portfolio

Final QA

Additional Features

These are not required, but they make the application feel complete.

Global Search

Ctrl + K

Search

- Chats
 - PDFs
 - Documents
-

Command Palette

Like VS Code.

Keyboard Shortcuts

N

New Chat

Ctrl + Enter

Send

Esc

Close

Toast

Every action.

Empty States

Beautiful.

Skeleton

Instead of spinner.

Markdown

Beautiful.

Code Highlight

Professional.

Mermaid

Optional.

Chat Export

Markdown

PDF

Copy

Theme

Dark

Light

System

API Status

Green

Yellow

Red

Backend

Database

AI

Things Codex is NEVER allowed to do

✗ Fake APIs

✗ Mock Data

✗ Hardcoded JSON

- ✗ Bootstrap
- ✗ Material UI
- ✗ Redux
- ✗ Giant Components
- ✗ Duplicate Code
- ✗ Random Folder Structure

I also want to add something that wasn't in our previous discussions

Every phase should finish with a **Quality Gate**.

Before moving to the next phase, we check:

✓ Functional

- Works with the backend.
- No broken features.

✓ Code Quality

- Clean architecture.
- No duplication.
- Strong TypeScript types.
- Small reusable components.

✓ UI/UX

- Responsive.
- Accessible.
- Smooth interactions.
- Consistent with the design system.

✓ Production Readiness

- No console errors.
- No TypeScript errors.
- No unused code.
- No hardcoded values.
- Environment variables used correctly.

Only when a phase passes all four gates do we move to the next one.